

Web A11y for Developers

Day 3: Accessible Forms

Agenda

Day 1: Introduction

Day 2: Components

Day 3: Accessible Forms

Day 4: Best Practices

Accessible forms

Forms are one of the **most important and complicated** pieces of any website or application - from an accessibility standpoint.

Building accessible forms takes an entire team, a number of **design decisions** to make and **development hurdles** to cross.

There are **a lot of moving parts**. Let's take a look at them.

Labeling

Every form has at least one input (text, checkbox, dropdown etc).
One thing is true about all of them - **they must have a label.**

Labels are the primary names of the form inputs and let users know that **“This is an input and this is what it’s called”**.

The only element we need is the **<label>** element.

Labeling

Explicit labeling

Using the **`for` attribute** on the `label` and connecting it with the **ID of the associated input**.

Example: When we cannot put them close together in the HTML.

Implicit labeling

Nesting the input inside the `<label>` element.

Example: When we need simple labeling, no need for IDs etc.

```
1 <!-- Explicit labeling (for + id) -->
2 <label for="first_name">First name</label>
3 <input id="first_name" type="text" placeholder="First name here..."></input>
4
5
6
7 <!-- Implicit labeling (nested input) -->
8 <label>
9   First name
10   <input type="text" placeholder="First name here..."></input>
11 </label>
```

Invisible labels

We can add invisible labels using the ``aria-label`` and ``aria-labelledby`` attributes.

There are scenarios where it is not feasible or possible to show a visible label for a number of reasons:

- **Spreadsheet interfaces** (Google Sheets, Excel)
- **URL search bars** on Browsers
- **Quick filters** where the **context around them** explains their role

index.html

```
1 <!-- Invisible labeling using aria-label (explicit label) -->
2 <input type="email" placeholder="Your email here.." aria-label="Email"></input>
3
4
5
6 <!-- Invisible labeling using aria-labelledby (more dynamic) -->
7 <p id="email">User email</p>
8 ...
9 <input type="email" placeholder="User email" aria-labelledby="email"></input>
```


“Web accessibility requires a context-based approach to solving problems.”

“There is no silver bullet solution to everything.”

- Anyone that's solved an a11y issue ever

Group labeling

Sometimes multiple inputs are logically associated with each other and they may require a common label associated with that entire section of inputs.

Example

Shipping address section that needs your **street name, number, city, zip code** etc.

We can achieve in 2 ways: **using the <fieldset> and <legend> elements** or **using roles and ARIA attributes**.

```
1 <!-- Group labeling: <fieldset> + <legend> -->
2 <form>
3   ...
4   <fieldset>
5     <legend>Shipping Address:</legend>
6
7     <label for="shipping_street">Street:</label>
8     <input type="text" id="shipping_street"></input>
9
10    <label for="street_number">Number:</label>
11    <input type="text" inputmode="numeric" id="street_number"></input>
12
13    <label for="zip_code">ZIP Code:</label>
14    <input type="text" inputmode="numeric" id="zip_code"></input>
15  </fieldset>
16  ...
17 </form>
```

```
1 <!-- Group labeling: roles + ARIA attributes -->
2 <form>
3   ...
4   <div role="group" aria-labelledby="shipping_address">
5     <p id="shipping_address">Shipping Address:</p>
6
7     <label for="shipping_street">Street:</label>
8     <input type="text" id="shipping_street"></input>
9
10    <label for="street_number">Number:</label>
11    <input type="text" inputmode="numeric" id="street_number"></input>
12
13    <label for="zip_code">ZIP Code:</label>
14    <input type="text" inputmode="numeric" id="zip_code"></input>
15  </div>
16  ...
17 </form>
```

Hints & Descriptions

Some inputs require **additional information alongside them**.

We may need to give users more context on:

- How should they fill in the information?
- How should that information be formatted?
- What is the meaning of that information?

This is what's called **“hint messages”** or **“descriptions”**.

We can add them using the **`aria-describedby` attribute**.

index.html

```
1 <label for="password">Password</label>
2 <input type="password" id="password" aria-describedby="password_hint"></input>
3
4 <!-- Screen readers will read this AFTER the input's label -->
5 <span id="password_hint">Must be at least 12 characters long.</span>
```

Errors & Live regions

In the event that **users enter invalid data**, or **the submission of our form fails** - we need to be able to tell them that there is a problem with the form.

Since **this information is not present on the form** when users start filling it in - **how do we notify them that an error happened?**

How do we tell screen readers that something's wrong with the form?

Live regions

Mechanism that allows assistive technologies to **announce changes on the page.**

What should we do?

Mark a section of the page as a live region.

How do we do that?

Either explicitly set an ``aria-live`` **attribute** or **specific roles with implicit live region attributes.**

Live regions

aria-live

Adding this attribute to an element **marks it as a live region**. Any changes to that element's content will be announced by ATs.

There are 3 ways this change can be announced:

1. **“assertive”** – announced **immediately**, interrupting the current read out of content
2. **“polite”** – announced **after the screen reader is done reading** other content
3. **“off”** – announced **only if the user has already moved focus within the live region**

Live regions

Roles as implicit live regions

The role of “**alert**” automatically makes an element a live region with an **assertive** setting.

The role of “**status**” automatically makes an element a live region with a **polite** setting.

Check out [some other roles that can do this too](#).

index.html

```
1 <label for="pwd">Password</label>
2 <input type="password" id="pwd" aria-describedby="error_msg hint_msg"></input>
3
4 <span id="hint_msg">Must be at least 12 characters long.</span>
5
6 <span aria-live="polite" id="error_msg">
7   ←!— Dynamically add an error here through JavaScript →
8 </span>
```

Error messages

Inline error messages

Appear **next to the input** that they are associated with.

Used for showing **descriptive errors**, often **offering a potential solution** within the message itself.

Can be used in different scenarios:

- As the user is **typing in the field** - can be distracting
- As soon as the user **leaves the field (on “blur”)** - can be confusing and distracting
- **After the form has been submitted** - generally good UX

Error messages

Summary error messages

Appear at the **start of the form**, within a **global live region**.

Used for showing a list of more **general error messages**.

They always appear **after the form has been submitted**.

```
1 <form>
2   <div role="alert">
3     <p>There was a problem sending your message!</p>
4
5     <ul>
6       <li>First and last name cannot be empty</li>
7       <li>You must provide a discount code</li>
8     </ul>
9   </div>
10
11   <label for="first_name">First name</label>
12   <input type="text" id="first_name">
13
14   <label for="last_name">Last name</label>
15   <input type="text" id="last_name">
16
17   <label for="discount_code">Discount code</label>
18   <input type="text" inputmode="numeric" id="discount_code">
19 </form>
```

Error messages

Best Practices

We always want to make error messages **distinguishable from the rest of the form, inputs and descriptions.**

Never rely on color only!

Get into the habit of showing an **error icon next to the message.**

We also need to make the error message content **clear and understandable.** Ideally include a way to solve the problem.

Break time!

We'll be right back.

Form validation

We can validate our forms either on the **client** or on the **server**. For security reasons, it is always a good idea to **validate it in both places**.

Required fields

HTML's built-in validation system allows us to put different constraints on inputs (min/max length, pattern, required etc.).

The **`required` attribute** (constraint) makes sure **an input must be filled with valid information** before it can be submitted.

index.html

```
1 <form>
2   <label for="name">Name</label>
3   <input required type="text" id="name" placeholder="Enter your full name...">
4
5   <button type="submit">Submit</button>
6 </form>
```

Form validation

Required fields (contd.)

The native ``required`` attribute works great with **native inputs, textareas, select inputs, checkboxes** etc.

Screen readers will announce the fact that a field is required.

For **custom input components**, we should use ``aria-required`` instead.

This will NOT trigger the HTML's built-in validation, but will make sure our component is announced as "required".

index.html

```
1 <form>
2   <label for="language">Language</label>
3   <custom-select aria-required="true" id="language">...</custom-select>
4
5   <button type="submit">Submit</button>
6 </form>
```

index.html

```
1 <form novalidate>
2   <label for="name">Name</label>
3   <input required type="text" id="name" placeholder="Enter your full name...">
4
5   <label for="language">Language</label>
6   <custom-select aria-required="true" id="language">...</custom-select>
7
8   <button type="submit">Submit</button>
9 </form>
```

Form submission

We always want to have **multiple ways our form can be submitted**.

The two most common ways:

- Having a **clickable submit button** at the bottom of the form
- Ability to submit the form by **pressing the `Enter` key**

We can easily enable both scenarios at the same time by **having the clickable submit button** have a **`type` of “submit”**.

index.html

```
1 <form novalidate>
2   <label for="email">Email</label>
3   <input required type="email" id="email" placeholder="Enter your email...">
4
5   <label for="message">Message</label>
6   <textarea type="text" id="message" placeholder="Enter your message...">
7
8   <button type="submit">Submit</button>
9 </form>
```

```
1 <form id="contact-form" novalidate>
2   <label for="email">Email</label>
3   <input required type="email" id="email" placeholder="Enter your email...">
4
5   <label for="message">Message</label>
6   <textarea type="text" id="message" placeholder="Enter your message...">
7 </form>
8
9 <!-- Some other UI here... -->
10
11 <button type="submit" form="contact-form">Submit</button>
```


Loading state

While our form is being submitted – we need to (at least) **disable the submit button** to avoid duplicated submissions.

Ideally we would also disable all of the inputs in the form.

Loading indicators

Generic spinners don't help users that rely on assistive technologies – **the loading state is not being announced** to them.

We want to **wrap our spinner inside a live region** and also add a “Loading” message (visible or invisible label).



index.html

```
1 <form>
2   <!-- Inputs + Submit button here -->
3
4   <div aria-live="polite">
5     <div class="spinner"></div>
6     <span>Loading... </span>
7   </div>
8 </form>
```

Loading state

Skeleton loading indicators

We need to mark these elements as **placeholder elements** that will tell assistive technologies that their content will change shortly.

How do we do that?

Set ``aria-busy="true"`` on the skeleton element.

Add an ``aria-label`` **attribute** describing what is being waited on.

``aria-busy``

Used to indicate that an **element's content is being modified**.

Content will be announced when its value becomes "false".

index.html

```
1 <form>
2   <!-- Inputs + Submit button here -->
3
4   <div aria-busy="true" aria-label="Loading user profile">
5     <div class="avatar-skeleton"></div>
6     <div class="username-skeleton"></div>
7   </div>
8 </form>
```

Success state

Probably **the most overlooked part** of accessible forms on the Web.

Remember, we want any any change in our form's state to be announced by assistive technologies.

Success messages don't need anything fancy, a **live region** will do just fine.



index.html

```
1 <form>
2 <!-- Inputs + Submit button here -->
3
4   <div aria-live="polite">
5     <span>Hurray! We got your message.</span>
6   </div>
7 </form>
```

Accessible forms

Recap

Make sure your inputs are **properly labeled**. Always try to have a visible label if possible.

Descriptions, hints and error messages need to be **programmatically connected** to their associated input.

Success/error messages and loading states need to be set up within live regions so that assistive technologies can announce them.

Always go with **native inputs, native attributes** if possible.

UX Improvements

The `autocomplete` attribute

This attribute helps browsers **understand the meaning** of our inputs.

They can then use their **autofill feature** to fill out our forms on behalf of our users.

The `autocomplete` attribute can contain many values, each for a different kind of input.

Note: The **autofill feature** can also come from other software such as **password managers**. The more the merrier!


```
1 <form>
2   <label for="cc_name">Name on card</label>
3   <input type="text" id="cc_name" autocomplete="cc-name">
4
5   <label for="cc_num">Card number</label>
6   <input type="text" inputmode="numeric" id="cc_num" autocomplete="cc-number">
7
8   <label for="cc_exp">Expiration date</label>
9   <input type="text" id="cc_exp" autocomplete="cc-exp">
10
11  <label for="cc_csc">Security code (CVV)</label>
12  <input type="text" inputmode="numeric" id="cc_csc" autocomplete="off">
13
14  <button type="submit">Pay now</button>
15 </form>
```

UX Improvements

The `autocomplete` attribute

This attribute has many perks:

- Users will **commit fewer errors** while submitting our forms
- Users with **cognitive, motor and similar disabilities** can fill out our forms more easily
- Screen readers will have more information about the form and will announce it more clearly
- Everyone will generally spend much less time filling out forms

UX Improvements

The `autocomplete` attribute

This is **NOT a silver bullet solution** for every input ever.

Notice that the CVV input field had **`autocomplete="off"`** - [PCI DSS compliance](#) prohibits storing users' CVV number on devices.

When to use `autocomplete="off"`

- **One-time tokens** and **password inputs** (sensitive information)
- **Discount/Promo code inputs** (expiring after usage)
- **Search inputs** where we list our own custom suggestions

UX Improvements

Input types & keyboard layouts

One of the most impactful ways to improve our forms - use the **appropriate input types and keyboard layouts**.

This helps browsers know **what kind of data is expected for a given input** and also provide the correct **virtual keyboard** (especially helpful for mobile devices).

UX Improvements

The `type` attribute

Tells browsers and assistive technologies **what kind of data an input expects**.

Common values: **“text”, “email”, “password”, “tel”** and **“number”**.

Choosing the right `type` can enable **browser autocomplete and validation** features (ex. `type="email"`).

index.html

```
1 <form>
2   <label for="email">Email</label>
3   <input type="email" id="email" autocomplete="email">
4
5   <label for="pwd">Password</label>
6   <input type="password" id="pwd" autocomplete="current-password">
7
8   <button type="submit">Log in</button>
9 </form>
```

UX Improvements

The ``inputmode`` attribute

Gives more fine-grained control over **which keyboard layout is displayed** and **which characters can be typed** into an input.

Common values:

- ``numeric`` - only digits (0-9)
- ``decimal`` - digits plus the decimal point
- ``tel`` - telephone keypad (0-9, *, #)
- ``email`` - keyboard layout optimized for emails (“@”, “.com”)
- ``url`` - keyboard layout optimized for URLs

UX Improvements

The ``inputmode`` attribute

This attribute can be particularly useful when the ``type`` attribute doesn't quite match the keyboard layout we want to show.

Common gotcha for ``type="number"``

Numeric inputs can be made to work better by combining ``type="text"`` and ``inputmode="numeric"``.

We want to use ``type="number"`` for **quantity based inputs** (ex. Age, Price, Items count etc.).

index.html

```
1 <form>
2   <label for="zip">ZIP Code</label>
3   <input type="text" id="zip" inputmode="numeric" autocomplete="postal-code">
4
5   <label for="houses">Max number of houses</label>
6   <input type="number" id="houses">
7
8   <button type="submit">Search</button>
9 </form>
```

Conclusion

There are a lot of moving parts when it comes to building accessible forms on the Web.

You are now equipped with the knowledge to tackle these challenges and direct your worries to the business logic, animation and styling.

Try to take these concepts and **apply them to your previous and future projects** – you might be surprised how easy it is to miss the small details.

Thank you!

See ya next time!